Tecnológico de Monterrey

httr2 y precauciones

Qué hacer cuando read_html() ya no alcanza

Ciencia de Datos II. Complemento de la Clase 16

Jorge Juvenal Campos Ferreira

Extraer datos reales exige más que read_html()

El raspado web (web scraping) extrae datos de páginas con código.

Situación

read_html() descargó sin problema los 133 artículos de Atiempo.

Complicación

El mismo método falló en gob.mx con un mensaje que no explicaba la causa.

Resolución

httr2 controla la petición, y seis precauciones protegen al sitio y a los datos.

Pregunta de la sesión

¿Qué cambia cuando el servidor deja de comportarse como esperamos?

En Atiempo, read_html() alcanzó para todo

El sitio entrega HTML normal y acepta la petición que envía R.

12

páginas recorridas

133

artículos extraídos

0

errores de conexión

```
html_juve <- read_html(url_juve)

titulos <- html_juve %>%
  html_nodes(".entry-title") %>%
  html_nodes("a") %>%
  html_text()
```

Cuando todo sale bien, una sola función oculta toda la conversación con el servidor.

Fuente: atiempo.tv, extracción propia del 4 de noviembre de 2026.

En gob.mx, la misma función falla sin explicar

El mensaje no dice si falló la URL, internet o el servidor.

```
pagina <- paste0(
  "https://www.gob.mx/presidencia/es/archivo/articulos",
  "?idiom=es&order=DESC&page=2"
)
codigo_pagina <- read_html(pagina)
```

```
Error in open.connection(x, "rb") :
  cannot open the connection
```

Lo que vimos en RStudio

La conexión no abrió, y R no mostró ningún código del servidor.

La pista que faltaba

Desde curl, la misma URL respondió con el código 200 (éxito).

El servidor rechaza al identificador de RStudio

El identificador (User-Agent) es el texto con el que el cliente se presenta.

Identificador enviado	Resultado
R (4.6.1 ...)	**Funciona**
Mozilla/5.0	**Funciona**
RStudio	**Funciona**
RStudio Desktop	**403**
RStudio Desktop (2026.7.1.147); R (...)	**403**

Qué significa

El código 403 indica que el servidor entendió la petición y se negó a atenderla. La regla exacta sigue sin conocerse.

Fuente: pruebas propias contra gob.mx, 4 de noviembre de 2026, con identificadores simulados.

Cada petición HTTP lleva datos que el servidor lee

HTTP es el protocolo de la web; req_dry_run() muestra la petición sin enviarla.

```
request(pagina) %>%
  req_user_agent("Mozilla/5.0") %>%
  req_dry_run()
```

```
GET /presidencia/es/archivo/articulos HTTP/1.1
accept: */*
accept-encoding: deflate, gzip
host: www.gob.mx
user-agent: Mozilla/5.0
```

GET

Este método pide un recurso al servidor.

host

Este campo nombra el sitio que recibe la petición.

user-agent

Este campo dice quién pregunta. Aquí estaba el problema.

Fuente: salida de httr2 1.3.0.

httr2 separa la petición en cuatro pasos

read_html() hace todo junto; httr2 te deja intervenir en cada paso.

request(url)	Prepara la petición con la URL. Todavía no envía nada.
req_user_agent(...)	Define el identificador con el que R se presenta.
req_perform()	Envía la petición y recibe estado, encabezados y cuerpo.
resp_body_string()	Saca el cuerpo de la respuesta como una cadena de texto.

Con httr2, el error ya dice qué pasó

El mismo rechazo se lee distinto en cada paquete.

read_html()

cannot open the connection

req_perform()

HTTP 403 Forbidden.

```
respuesta <- request(pagina) %>%
  req_user_agent(identificador_rstudio) %>%
  req_error(is_error = function(resp) FALSE) %>%
  req_perform()

resp_status(respuesta)   # 403
resp_status_desc(respuesta)   # "Forbidden"
```

Un diagnóstico preciso evita buscar la falla en el lugar equivocado.

Fuente: pruebas propias con httr2 1.3.0 y rvest; 403 definido en RFC 9110.

Una respuesta 200 no garantiza que sea HTML

Con el identificador resuelto apareció un segundo problema.

```
<httr2_response>
Status: 200 OK
Content-Type: text/javascript
Body: In memory (10863 bytes)
```

```
$('#prensa').append('<div class="row clearfix
bottom-buffer">  <div class=\"col-md-4\">\n
<article>\n ...
```

Qué pasa con read_html()

No marca error, pero los enlaces salen como \"/presidencia/...\" y los títulos traen basura.

Qué conviene revisar

Content-Type dice qué trae el cuerpo, y resp_content_type() lo consulta en httr2.

Fuente: gob.mx, extracción propia del 4 de noviembre de 2026.

Tres reemplazos recuperan el HTML escondido

resp_body_string() entrega el texto para limpiarlo antes de read_html().

En la respuesta	Se convierte en
\"	"
\/	/
\n	un espacio

```
html_pagina <- texto_pagina %>%
  str_replace_all('\\\\"', '"') %>%
  str_replace_all("\\\\/", "/") %>%
  str_replace_all("\\\\n", " ") %>%
  read_html()
```

Resultado

Cada página del listado entregó 9 artículos con título, fecha y enlace limpios.

Fuente: gob.mx, extracción propia del 4 de noviembre de 2026.

Las pausas protegen al sitio y a tu acceso

Un bucle sin pausas dispara peticiones tan rápido como puede.

125

peticiones en total:
25 de listado y 100 de texto

125 s

de espera como mínimo,
con 1 segundo de pausa

```
for (i in 1:nrow(tabla_conferencias)) {
  # ... descargar y extraer el texto ...
  Sys.sleep(1)
}
```

Por qué importa

El servidor atiende también a otras personas. Además, un ritmo agresivo puede provocar que el sitio bloquee tu dirección.

Fuente: scraping_sheinbaum.R.

Filtrar exige revisar lo que se descarta

El listado mezcla conferencias con giras y otros eventos.

225

artículos revisados en 25 páginas

103

títulos con "conferencia de prensa"

100

conferencias más recientes conservadas

Lo que falló en el primer intento

El patrón largo dejó fuera este título: "Conferencia de prensa matutina del 21 de agosto de 2026".

```
filter(str_detect(
  titulo,
  regex("conferencia de prensa", ignore_case = TRUE)
))
```

Fuente: gob.mx, páginas 1 a 25 del archivo, extracción propia del 4 de noviembre de 2026.

Validar evita confiar en datos incompletos

Un script que termina sin error todavía puede estar mal.

```
stopifnot(
  nrow(tabla_final) == numero_conferencias,
  !any(is.na(tabla_final$texto))
)
```

100 filas

El script reunió las conferencias pedidas.

0 textos vacíos

Todas las conferencias traen su versión.

0 fechas repetidas

Ninguna conferencia aparece dos veces.

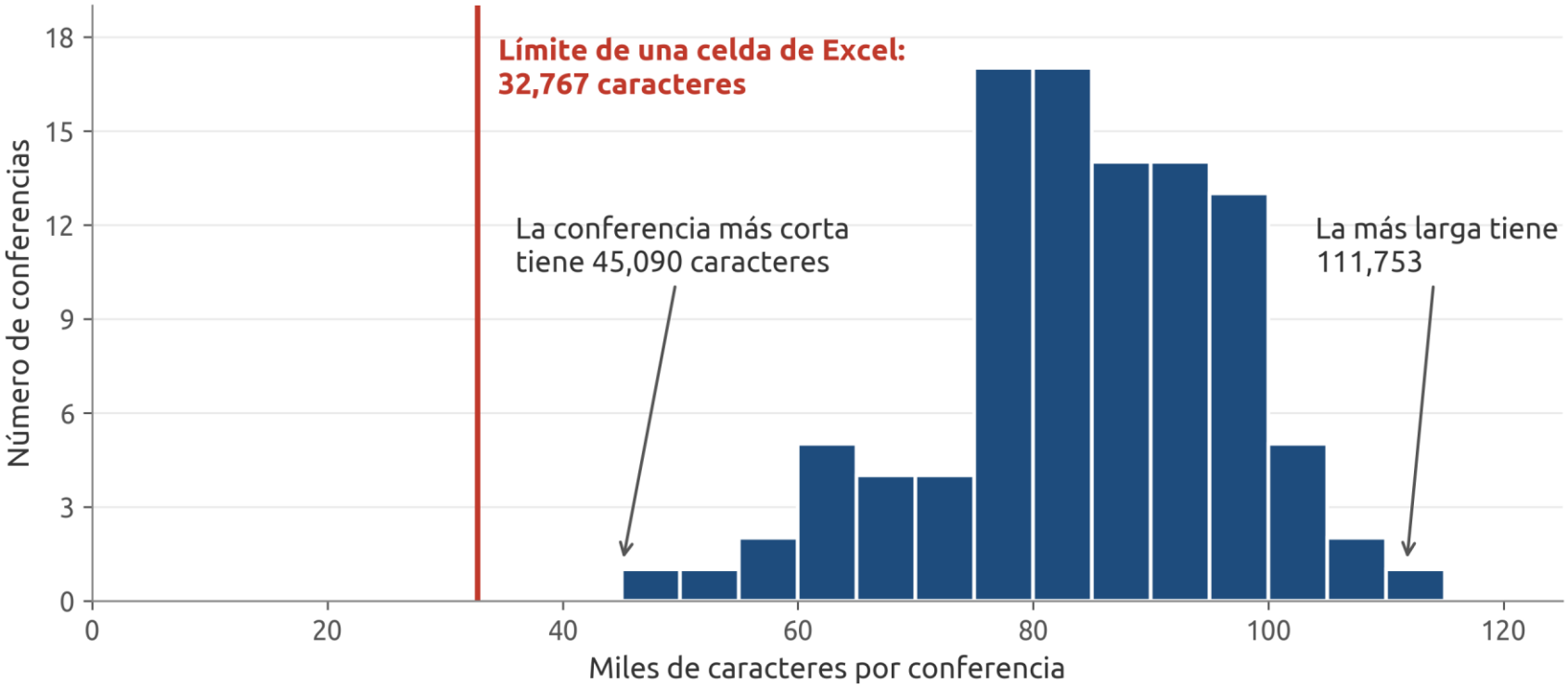
Falta confirmar con otra fuente

Cuatro días hábiles no aparecen: 1, 15 y 16 de septiembre y 2 de noviembre de 2026.

Fuente: gob.mx, extracción propia del 4 de noviembre de 2026.

Las 100 conferencias rebasan el límite de Excel

Por eso el script guarda en CSV y se lee con read_csv().



Atiempo sí cabe en xlsx: su texto más largo ronda 16 mil caracteres.

Fuente: gob.mx, cálculos propios. Límite: Microsoft (s. f.).

Cambiar el identificador también tiene costo ético

La información es pública, pero el filtro lo puso el sitio.

Lo que hicimos

Enviamos otro identificador y con eso esquivamos un filtro del sitio.

Lo que no sabemos

Desconocemos si el sitio quiso bloquear a RStudio o si la regla lo alcanza por accidente.

Lo que conviene revisar antes

Revisa robots.txt: el de gob.mx solo menciona rastreadores de Google.
Lee los términos de uso del sitio.
Busca primero una API o datos abiertos oficiales.
Detente si el sitio pide inicio de sesión o CAPTCHA (prueba contra robots).

Fuente: gob.mx/robots.txt; Koster et al. (2022).

MercadoLibre obliga a buscar otra vía

La misma pregunta técnica recibe otra respuesta en un sitio comercial.

Criterio	gob.mx	MercadoLibre
robots.txt	Solo menciona rastreadores de Google	Veta a todos _Desde_ y _NoIndex_True
Sitio	Gobierno con información pública	Comercio con anuncios de terceros
Raspado	Sin regla general en robots.txt	Declara medidas para impedirlo
API	No se revisó	Exige aplicación registrada y token
Decisión	**Raspar con pausas**	**No raspar: usar API o pedir permiso**

Una API con token no garantiza los datos

Fuentes de terceros reportan que la búsqueda general sigue restringida aun con token. Falta confirmarlo con una aplicación registrada.

Fuente: robots.txt de ambos sitios, 4 de noviembre de 2026; Mercado Libre (s. f.-a, s. f.-b).

httr2 ya incluye pausas y reintentos

Estas funciones reemplazan el Sys.sleep() manual.

```
respuesta <- request(url_base) %>%
  req_url_query(idiom = "es", order = "DESC", page = k) %>%
  req_user_agent("Mozilla/5.0") %>%
  req_throttle(capacity = 1, fill_time_s = 1) %>%
  req_retry(max_tries = 3) %>%
  req_perform()
```

req_url_query()

Arma los parámetros de la URL sin pegar cadenas a mano.

req_throttle()

Limita el ritmo. Así, 25 páginas tardaron 24.5 segundos.

req_retry()

Repite la petición hasta tres veces si ocurre una falla pasajera.

Fuente: prueba propia con httr2 1.3.0, 4 de noviembre de 2026.

Cada precaución evita un riesgo concreto

Las seis precauciones del script responden a seis riesgos.

Precaución	Riesgo que evita
Fijar el identificador	Recibir un rechazo 403
Revisar Content-Type	Extraer basura sin ver un error
Pausar entre peticiones	Saturar el sitio o perder acceso
Auditar el filtro	Perder registros en silencio
Validar con stopifnot()	Confiar en datos incompletos
Guardar en CSV	Cortar textos largos en Excel

Estas fuentes respaldan lo visto en la sesión

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). RFC Editor. https://www.rfc-editor.org/rfc/rfc9110

Koster, M., Illyes, G., Zeller, H., & Sassman, L. (2022). *Robots Exclusion Protocol* (RFC 9309). RFC Editor. https://www.rfc-editor.org/rfc/rfc9309

Mercado Libre. (s. f.-a). *Autenticación y autorización*. Mercado Libre Developers. https://developers.mercadolibre.com.ar/es_ar/autenticacion-y-autorizacion

Mercado Libre. (s. f.-b). *¿Cómo pueden usar otros usuarios tus datos?* https://www.mercadolibre.com.ar/privacidad/declaracion-privacidad/31256

Microsoft. (s. f.). *Excel specifications and limits*. Microsoft Support. https://support.microsoft.com/en-us/office/excel-specifications-and-limits-1672b34d-7043-467e-8e27-269d656771c3

Wickham, H. (2025). *rvest: Easily harvest (scrape) web pages* (versión 1.0.5) [Software]. https://CRAN.R-project.org/package=rvest

Wickham, H. (2026). *httr2: Perform HTTP requests and process the responses* (versión 1.3.0) [Software]. https://CRAN.R-project.org/package=httr2

Conclusiones: el raspado es también diagnóstico

1. read_html() basta cuando el sitio entrega HTML y acepta tu petición.
2. httr2 separa la petición y muestra por qué falló.
3. Un código 200 no garantiza que recibiste el contenido esperado.
4. Las pausas y las revisiones de términos protegen al sitio y a tu trabajo.
5. Validar los datos y auditar los filtros detecta errores silenciosos.

Gracias

¿Preguntas?